AULA 07

CSS

cascata, formas de aplicação e seletores

Controlar a aparência e entender qual regra vence.

07

Se duas regras mudam a mesma cor, qual delas vence?

```css
p { color: blue; }
```

```css
.aviso { color: red; }
```

```html
<p class="aviso">Mensagem</p>
```

> **A resposta depende da cascata e da especificidade.**

Objetivos de aprendizagem

01 **Ler uma regra**

seletor, propriedade e valor

02 **Aplicar CSS**

inline, interno e externo

03 **Prever conflitos**

cascata, ordem e especificidade

04 **Escolher seletores**

simples, combinados e por estado

05 **Explicar o resultado**

código junto da tela produzida

06 **Escrever com clareza**

regras reutilizáveis e fáceis de revisar

Caminho da aula

01

Fundamentos

regra e declaração

02

Aplicação

inline, interno, externo

03

Cascata

conflitos e herança

04

Seletores

quem recebe o estilo

05

Aplicação

atividade e revisão

As questões de verificação ficam reunidas no final.

HTML e CSS têm responsabilidades diferentes

HTML

Descreve o conteúdo e sua estrutura: título, parágrafo, link, imagem e formulário.

CSS

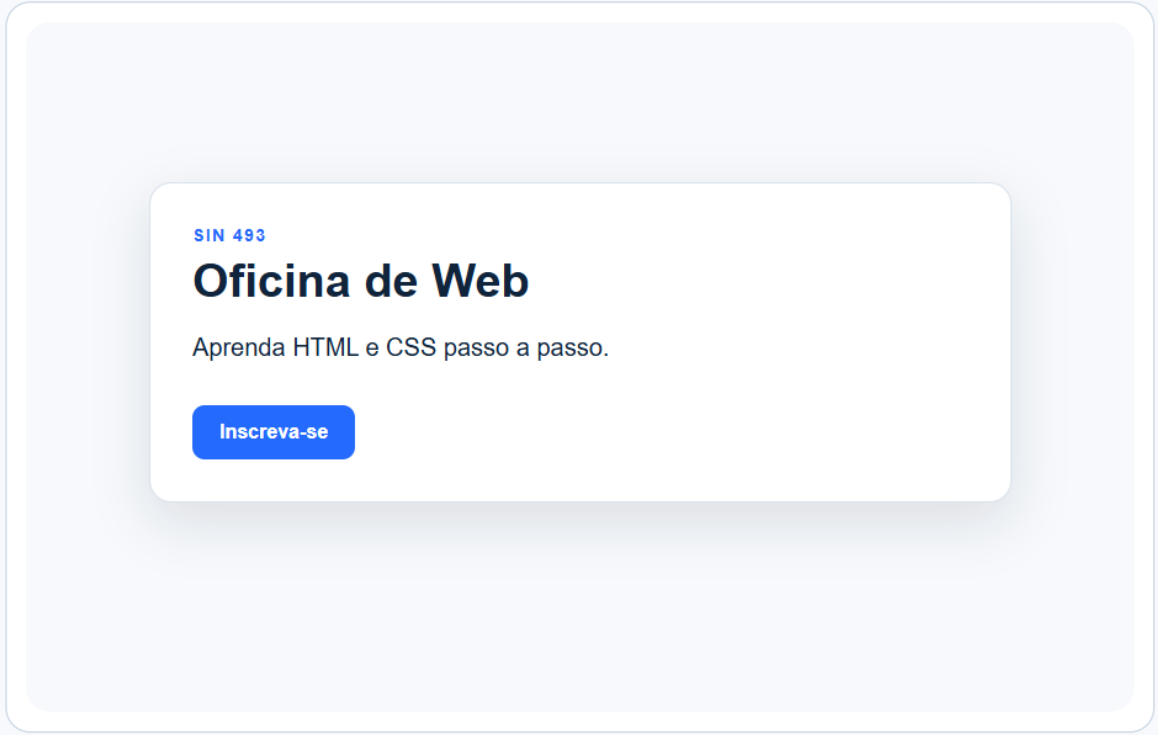
Descreve a apresentação: cores, tamanhos, espaços, bordas e posicionamento.

O navegador combina a estrutura do HTML com as regras do CSS.

A mesma página antes e depois do CSS



SEM CSS DO AUTOR



COM CSS DO AUTOR

> **O conteúdo continua; muda a forma de apresentá-lo.**

Fundamentos do CSS

Como uma regra encontra elementos e define sua aparência.

01

CSS descreve como o conteúdo aparece

CSS significa Cascading Style Sheets, ou Folhas de Estilo em Cascata.
Uma folha de estilo contém regras.
Cada regra seleciona elementos do HTML.
As declarações informam o que deve mudar nesses elementos.
O navegador calcula um valor final para cada propriedade.

Anatomia de uma regra CSS

CSS

```
p {
  color: #246bfd;
  font-size: 20px;
}
```

Uma regra completa

p é o seletor. O trecho entre chaves é o bloco de declarações. Cada linha tem propriedade, dois-pontos, valor e ponto e vírgula.

Leia o seletor e depois cada declaração.

O seletor aponta para elementos

CSS

```css
h1 {
  color: #10243c;
}
```

Quem recebe a regra

O seletor h1 corresponde a todos os elementos <h1> do documento. Apenas os elementos que combinam com o seletor entram na disputa da cascata.

Leia o seletor e depois cada declaração.

A declaração une propriedade e valor

CSS

```
p {
  color: blue;
}
```

O que deve mudar

color é a propriedade. blue é o valor. Os dois-pontos separam os dois. O ponto e vírgula encerra a declaração.

Leia o seletor e depois cada declaração.

Um bloco pode ter várias declarações

CSS

```css
p {
  color: #10243c;
  font-size: 20px;
  line-height: 1.5;
  max-width: 60ch;
}
```

Uma ideia por linha

Cada declaração altera uma característica. Escrever uma por linha facilita leitura, correção e comparação.

Leia o seletor e depois cada declaração.

Comentários explicam decisões

CSS

```css
/* Limita a largura para facilitar a leitura
*/
p {
  max-width: 60ch;
}
```

Texto ignorado pelo CSS

Um comentário começa com /* e termina com */. Ele deve explicar uma decisão útil, não repetir o que o código já diz.

Leia o seletor e depois cada declaração.

Propriedades iniciais para observar

`color`

cor do texto

`background-color`

cor do fundo

`font-size`

tamanho da fonte

`font-family`

família tipográfica

`line-height`

altura da linha

`margin`

espaço externo

`padding`

espaço interno

`border`

contorno do elemento

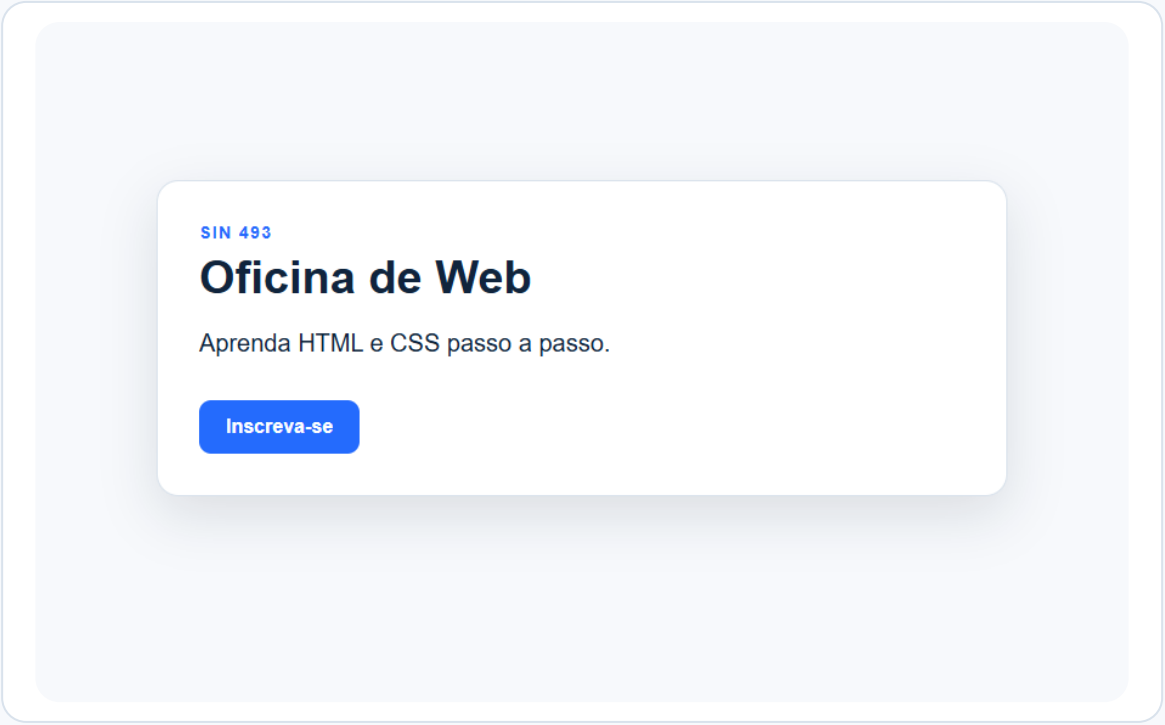
> **O nome da propriedade informa qual característica será calculada.**

Regra simples e resultado

HTML + CSS

```
HTML
<h1>Oficina de Web</h1>
<p>Aprenda HTML e CSS passo a passo.</p>

CSS
h1 { color: #10243c; font-size: 38px; }
p { font-size: 20px; line-height: 1.45; }
```



O navegador aplica cada declaração aos elementos selecionados.

Separar conteúdo e apresentação

O HTML conserva o significado e a estrutura do conteúdo.
O CSS concentra decisões visuais.
Uma mesma regra pode atender muitos elementos.
Uma alteração no CSS pode atualizar várias páginas.
A separação reduz repetição e facilita manutenção.

Onde escrever CSS

Estilos inline, internos e externos.

02

CSS inline usa o atributo style

HTML

```
<p style="color: #d94b3d; font-size: 30px;">
  Aviso importante
</p>
```

Diretamente no elemento

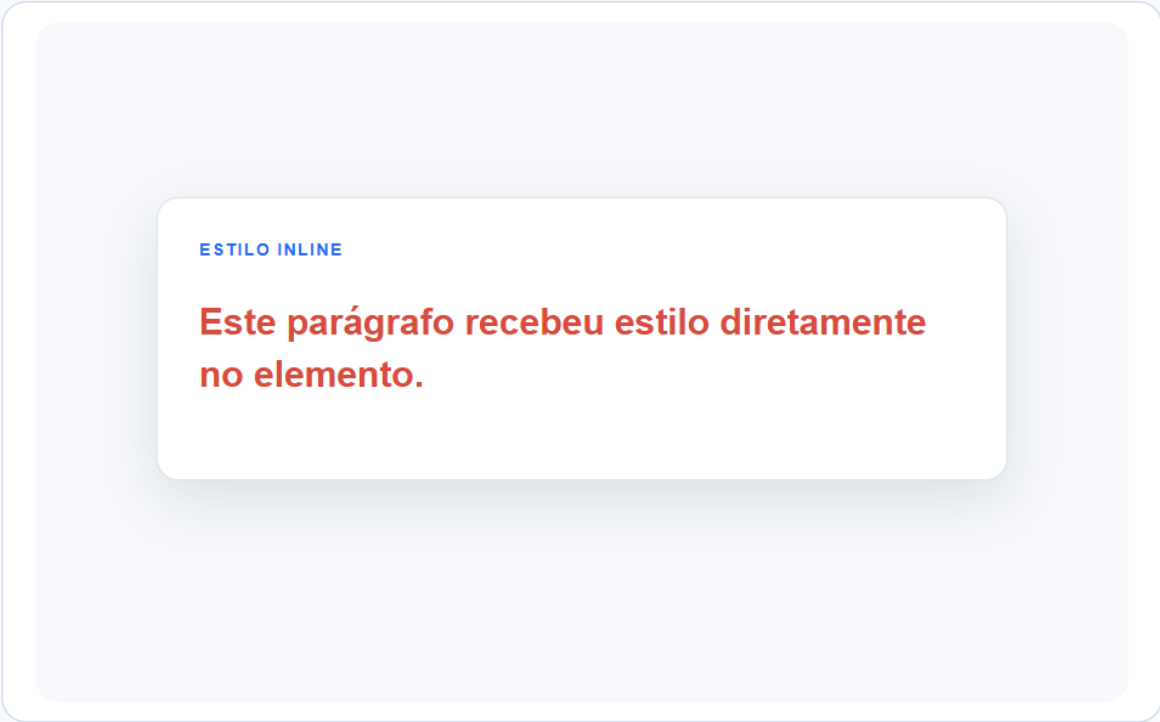
style pertence ao elemento HTML. Dentro do atributo, escrevemos declarações sem seletor e sem chaves.

Leia o seletor e depois cada declaração.

Inline no código e na tela

HTML

```
<p style="color: #d94b3d;
          font-size: 30px;
          font-weight: 700;">
  Este parágrafo recebeu estilo
  diretamente no elemento.
</p>
```



O atributo afeta apenas o elemento em que foi escrito.

Quando o inline ajuda e quando atrapalha

Uso pontual

Pode ajudar em um teste rápido ou quando um sistema gera um valor específico para um elemento.

Manutenção difícil

Repete declarações, mistura estrutura e aparência e tem alta prioridade na cascata normal.

Para sites comuns, prefira regras reutilizáveis em uma folha de estilo.

CSS interno usa <style> no head

HTML + CSS

```
<head>
  <meta charset="UTF-8">
  <title>Oficina</title>
  <style>
    h2 { color: #20a8b8; }
    p { line-height: 1.5; }
  </style>
</head>
```

Regras da própria página

style contém regras CSS. O navegador aplica essas regras ao documento atual. O bloco costuma ficar no head.

Leia o seletor e depois cada declaração.

CSS interno no código e na tela

HTML + CSS

```
<style>
  h2 { color: #20a8b8; }
  p { font-size: 20px; }
</style>

<h2>Uma regra dentro de &lt;style&gt;</h2>
<p>Os elementos selecionados recebem a
declaração.</p>
```



O bloco interno pode estilizar muitos elementos da mesma página.

Alcance do CSS interno

Vantagem

Mantém as regras reunidas e permite estilizar vários elementos sem repetir atributos.

Limite

As regras pertencem apenas àquela página. Repetir o mesmo bloco em dez arquivos cria cópias.

Use quando o estilo pertence somente ao documento ou durante um exercício curto.

CSS externo usa <link>

HTML

```
<head>
  <meta charset="UTF-8">
  <title>Oficina</title>
  <link rel="stylesheet" href="css/estilos.css">
</head>
```

Liga HTML e arquivo CSS

link é um elemento vazio. rel="stylesheet" informa a relação; href indica o caminho do arquivo CSS.

Leia o seletor e depois cada declaração.

O arquivo externo contém apenas CSS

CSS

```
/* css/estilos.css */
body {
  font-family: Arial, sans-serif;
  background-color: #f7f9fc;
}

h2 {
  color: #246bfd;
}
```

Sem tags HTML

O arquivo .css recebe regras CSS. Não escreva <style> dentro dele. O comentário identifica o arquivo apenas para a explicação.

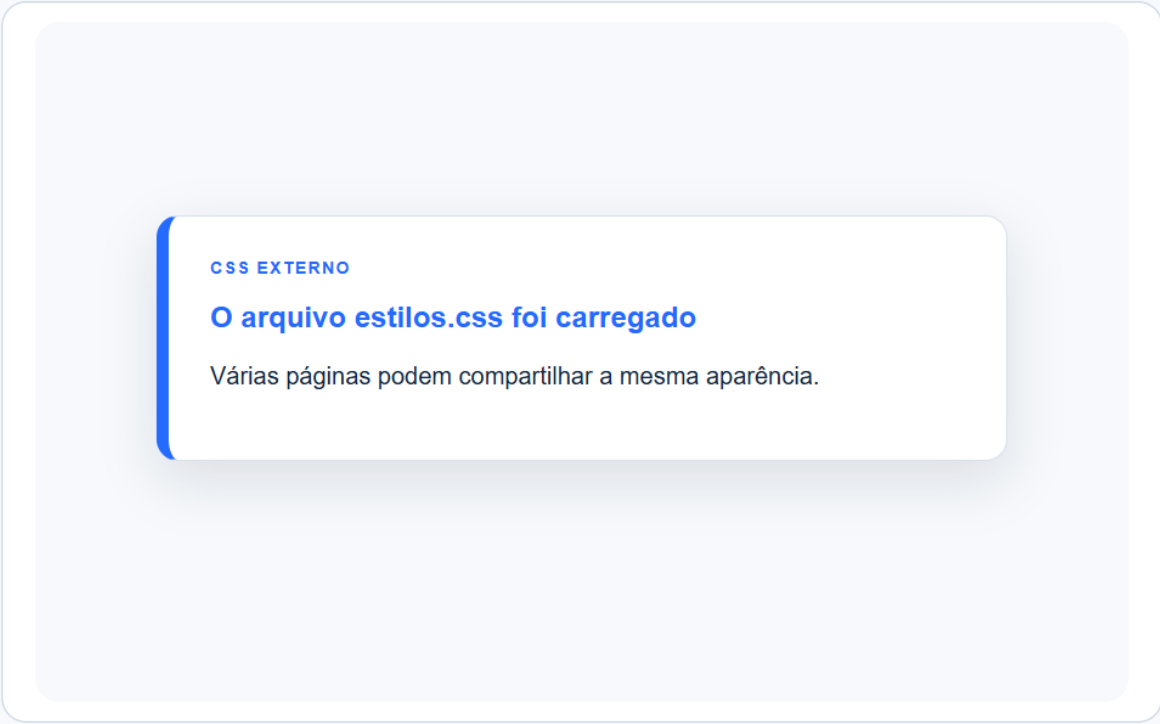
Leia o seletor e depois cada declaração.

CSS externo no código e na tela

HTML + CSS

```
HTML
<link rel="stylesheet" href="css/estilos.css">
<h2>O arquivo estilos.css foi carregado</h2>
<p>Várias páginas podem compartilhar a aparência.</p>

CSS
h2 { color: #246bfd; }
```



Um arquivo externo pode atender todas as páginas do site.

Caminhos do arquivo CSS

PASTAS

```
projeto/
  index.html
  css/
    estilos.css
  paginas/
    contato.html
```

HTML

```
<!-- em index.html -->
<link rel="stylesheet" href="css/estilos.css">

<!-- em paginas/contato.html -->
<link rel="stylesheet" href="../css/estilos.css">
```

O caminho é calculado a partir do arquivo HTML atual.

Comparação das três formas

Forma	Onde fica	Alcance	Uso comum
Inline	atributo style	um elemento	exceção pontual
Interno	tag style no head	uma página	atividade ou página única
Externo	arquivo .css ligado por link	várias páginas	site e manutenção

O CSS externo costuma oferecer a melhor reutilização.

Erros comuns ao aplicar CSS

href incorreto

O navegador não encontra o arquivo.

rel ausente

A relação com a folha não fica declarada.

<style> no .css

O arquivo externo recebe regras, não tags.

Seletor sem alvo

Nenhum elemento combina com a regra.

Propriedade digitada errado

A declaração inválida é ignorada.

Cache do navegador

Recarregue e confira o arquivo efetivamente servido.

Cascata

Como o navegador resolve regras que competem.

03

Cascata é o algoritmo de decisão

Várias declarações podem definir a mesma propriedade para o mesmo elemento.
O navegador elimina regras que não se aplicam.
Depois compara origem, importância e camadas.
Se ainda houver empate, compara especificidade e proximidade de escopo.
Quando todo o restante empata, a declaração que aparece por último vence.

Duas regras podem competir

CSS + HTML

```
p {
  color: blue;
}

.aviso {
  color: red;
}

<p class="aviso">Mensagem</p>
```

As duas regras combinam

p encontra o elemento pela tag. .aviso encontra o mesmo elemento pela classe. As duas definem color, então o navegador precisa escolher um valor.

Leia o seletor e depois cada declaração.

Modelo prático para analisar um conflito

1 **A regra se aplica?** O seletor encontra o elemento e a condição é relevante?

2 **Há !important?** Importância e origem entram antes da especificidade.

3 **Qual seletor pesa mais?** Compare ID, classe/atributo/estado e tipo.

4 **Existe style inline?** Declarações no atributo têm prioridade sobre regras normais do autor.

5 **Tudo empatou?** A declaração posterior vence.

Visão completa da ordem da cascata

1	**Relevância**	a regra se aplica ao elemento e ao contexto
2	**Origem e importância**	navegador, usuário, autor e !important
3	**Contexto e style**	encapsulamento e declarações ligadas ao elemento
4	**Camadas**	ordem definida por @layer
5	**Especificidade**	peso do seletor
6	**Proximidade de escopo**	distância dentro de @scope
7	**Ordem de aparição**	critério final de desempate

Estilos do navegador e estilos do autor

Navegador

Fornece estilos padrão, como margens em títulos e marcadores em listas.

Autor

As regras criadas pelo desenvolvedor normalmente substituem estilos normais do navegador.

A cascata também preserva estilos importantes escolhidos pelo usuário.

Style inline e regras normais do autor

CSS + HTML

```
p {
  color: blue;
}

<p style="color: red;">Mensagem</p>
```

O inline vence neste caso

A declaração ligada diretamente ao elemento tem prioridade sobre regras normais mapeadas por seletores do autor.

Leia o seletor e depois cada declaração.

Ordem de aparição como desempate

HTML + CSS

```
.mensagem {
  color: blue;
}

.mensagem {
  color: #ff6b5b;
}

<p class="mensagem">A segunda regra venceu.</p>
```



Mesma origem, importância e especificidade: a regra posterior vence.

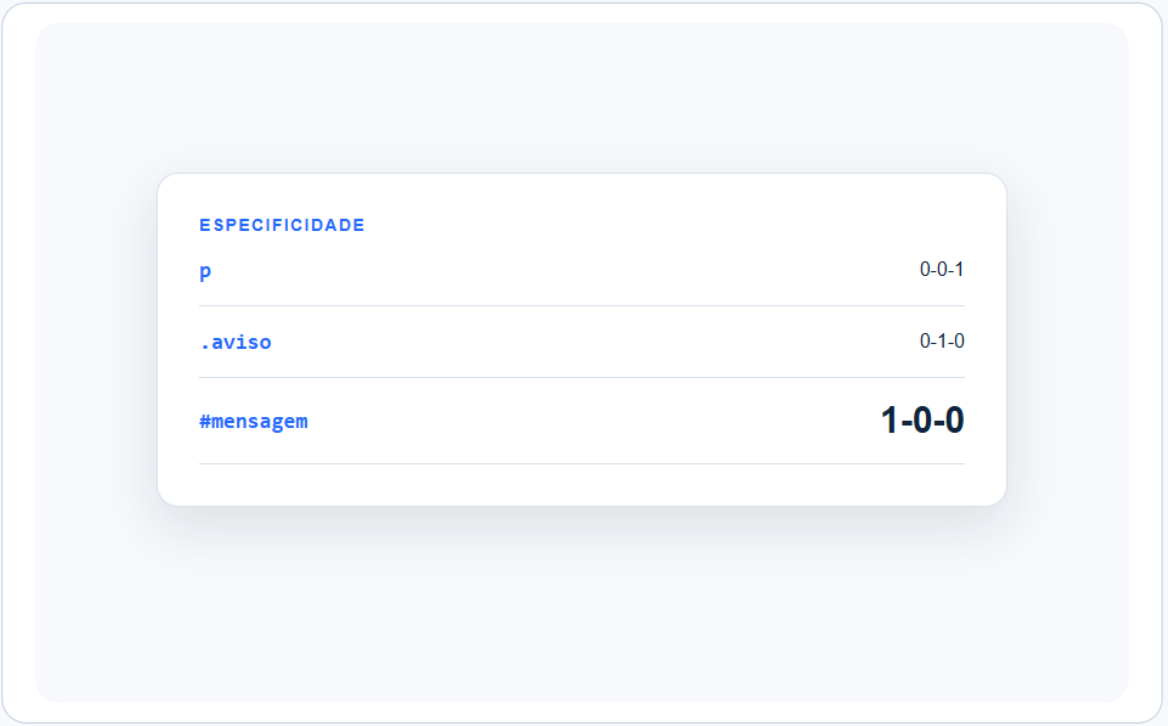
Especificidade mede o peso do seletor

A especificidade só é comparada entre declarações que continuam competindo.
ID acrescenta peso na primeira coluna.
Classe, seletor de atributo e pseudo-classe acrescentam peso na segunda coluna.
Tipo de elemento e pseudo-elemento acrescentam peso na terceira coluna.
O seletor universal e os combinadores não acrescentam peso.

Pesos básicos de especificidade

SELETORES

```
p              /* 0-0-1 */
.aviso         /* 0-1-0 */
#mensagem      /* 1-0-0 */
p.aviso       /* 0-1-1 */
main p.aviso /* 0-1-2 */
```



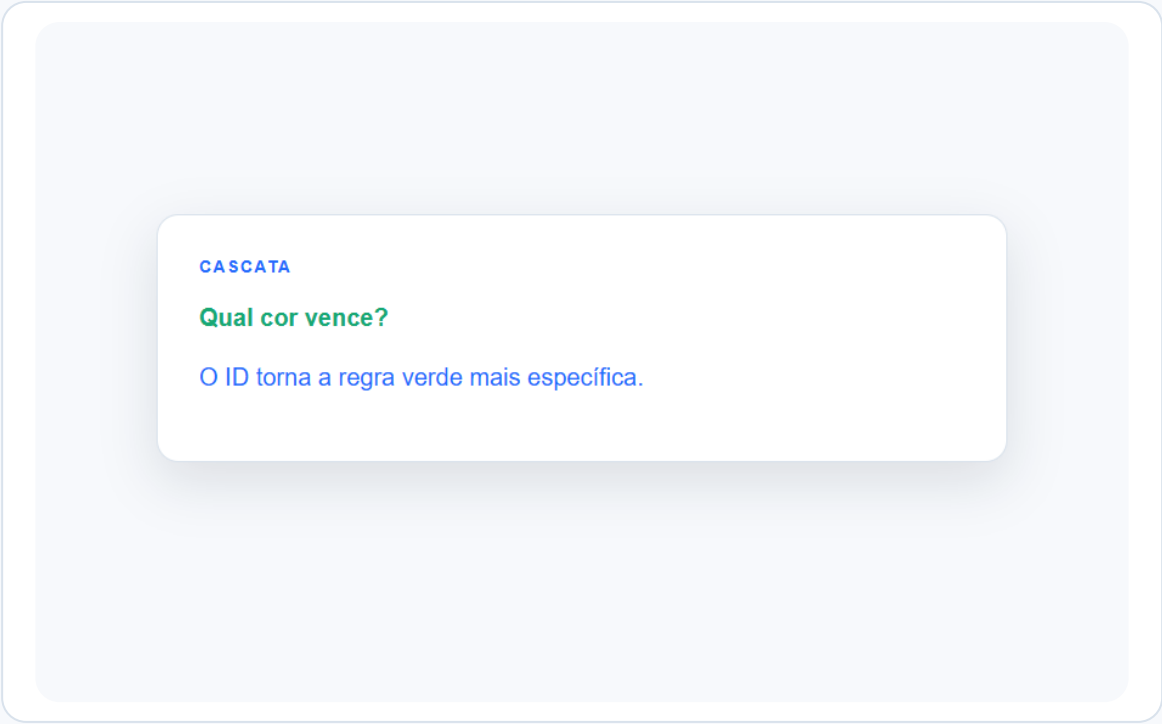
Compare ID, depois classe e por último tipo.

Tipo, classe e ID em conflito

CSS + HTML

```
p { color: #246bfd; }
.aviso { color: #ff6b5b; }
#mensagem { color: #18a874; }

<p id="mensagem" class="aviso">
  Qual cor vence?
</p>
```



O seletor #mensagem tem maior especificidade e torna o texto verde.

!important altera a etapa de importância

CSS

```css
.aviso {
  color: red !important;
}

#mensagem {
  color: green;
}
```

A regra importante vence

Dentro da mesma origem e camada, uma declaração importante vence uma declaração normal, mesmo quando o seletor normal possui maior especificidade.

Leia o seletor e depois cada declaração.

Uso consciente de !important

Pode ser necessário

Uma regra excepcional e documentada pode proteger um requisito ou sobrescrever CSS que não pode ser alterado.

Pode gerar disputa

Uso frequente cria regras cada vez mais difíceis de substituir e compreender.

Primeiro verifique seletor, ordem, camada e estrutura do CSS.

Herança preenche valores ausentes

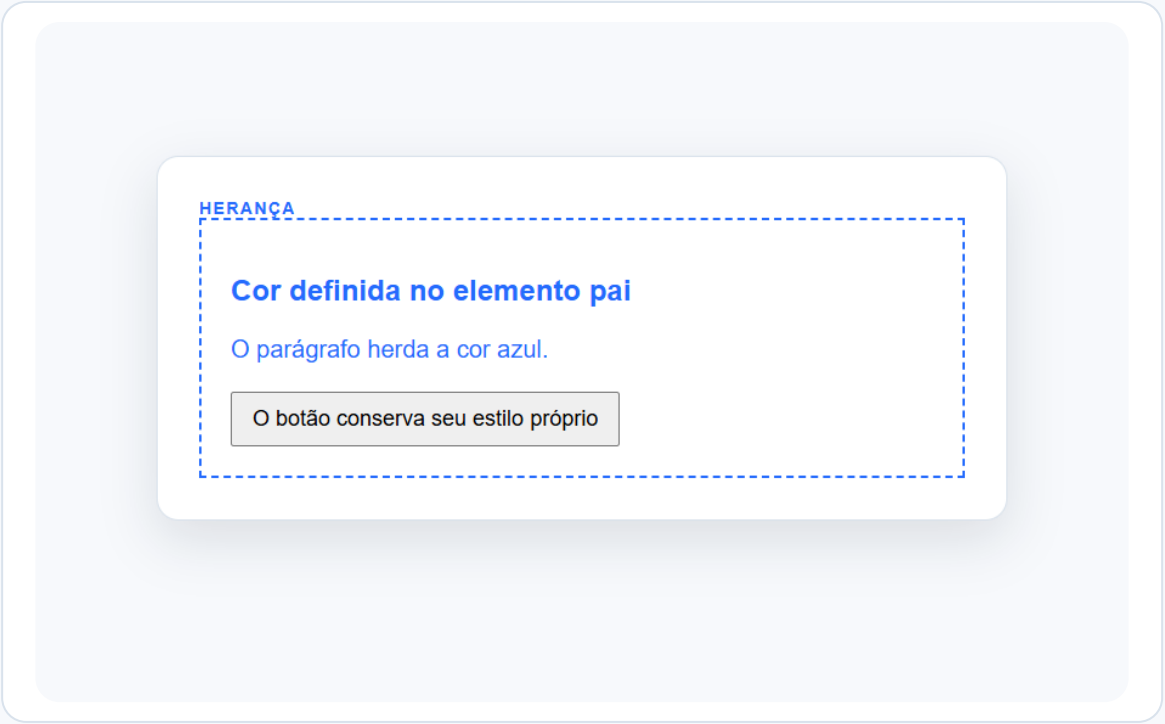
Algumas propriedades, como color, costumam passar do elemento pai para os descendentes.
Outras propriedades, como width e border, normalmente não são herdadas.
A herança atua quando o elemento não possui um valor vencedor diretamente aplicado.
Uma regra que seleciona o filho vence um valor apenas herdado do pai.

Herança no código e na tela

HTML + CSS

```
.area {
  color: #246bfd;
}

<div class="area">
  <h2>Cor definida no pai</h2>
  <p>O parágrafo herda a cor.</p>
  <button>Estilo próprio</button>
</div>
```



h2 e p recebem color do elemento pai; o controle conserva estilos próprios.

Cascata, especificidade e herança

Cascata

ordena declarações concorrentes

Especificidade

compara o peso de seletores dentro da disputa

Herança

obtém certos valores a partir do elemento pai

Os três conceitos explicam grande parte dos resultados inesperados.

Ferramentas do navegador mostram a regra vencedora

Inspecione o elemento na página.
Abra a lista de estilos aplicados.
Regras riscadas perderam uma disputa ou foram substituídas.
Confira o arquivo e a linha de origem.
Teste uma declaração temporariamente antes de editar o arquivo.

Erros comuns ao interpretar a cascata

A última sempre vence

Somente quando os critérios anteriores empatam.

ID usado em todo lugar

Cria especificidade alta e reduz reutilização.

!important como rotina

Esconde a causa do conflito.

Herança confundida com cascata

Valor herdado não é uma regra direta no filho.

Seletor não combina

A regra nem entra na disputa.

Arquivo não carregou

Nenhuma regra do arquivo pode participar.

Seletores

Como indicar quais elementos recebem cada regra.

04

Seletor universal: *

CSS

```css
* {
  box-sizing: border-box;
}
```

Todos os elementos

O asterisco combina com qualquer elemento. Neste exemplo, width e height passam a incluir conteúdo, padding e border no cálculo da caixa.

Leia o seletor e depois cada declaração.

Seletor de tipo: p

CSS

```css
p {
  color: #10243c;
  line-height: 1.5;
}
```

Elementos pelo nome da tag

A regra corresponde a todos os elementos <p>. É útil para estilos gerais de um tipo de conteúdo.

Leia o seletor e depois cada declaração.

Seletor de classe: .destaque

CSS + HTML

```
.destaque {
  background-color: #eaf1ff;
  border: 2px solid #246bfd;
}

<article class="destaque">...</article>
```

Elementos que compartilham uma classe

O ponto faz parte da sintaxe do seletor. No HTML, class recebe apenas o nome destaque, sem ponto.

Leia o seletor e depois cada declaração.

Um elemento pode ter várias classes

CSS + HTML

```
.cartao { padding: 24px; }
.aviso { border-left: 6px solid #ff6b5b; }

<article class="cartao aviso">
  Prazo encerrado
</article>
```

Classes separadas por espaço

O article recebe as regras de .cartao e .aviso. Cada classe representa uma característica reutilizável.

Leia o seletor e depois cada declaração.

Seletor de ID: #principal

CSS + HTML

```
#principal {
  background-color: #fff0ec;
}

<main id="principal">...</main>
```

Um identificador específico

A cerquilha faz parte do seletor. No HTML, id recebe o nome sem cerquilha. IDs devem ser únicos no documento.

Leia o seletor e depois cada declaração.

Elemento, classe e ID na mesma página

HTML + CSS

```
article { background: #edf2f7; }
.destaque { border: 2px solid #246bfd; }
#principal { border: 2px solid #ff6b5b; }
```



O seletor escolhido controla alcance, reutilização e especificidade.

Agrupamento evita repetir declarações

HTML + CSS

```
h2,
p,
a {
  color: #246bfd;
}
```



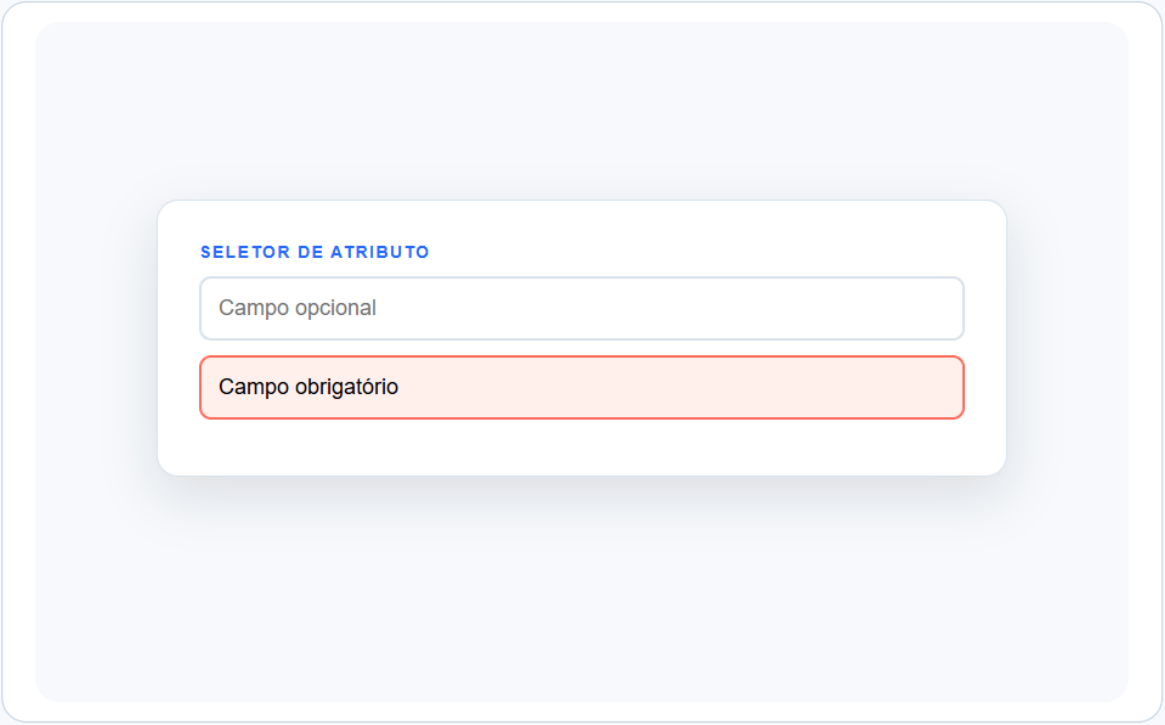
A vírgula separa seletores que compartilham o mesmo bloco.

Seletor de atributo

CSS + HTML

```
input[required] {
  border-color: #ff6b5b;
  background-color: #fff0ec;
}

<input type="email" required>
```



A regra encontra inputs que possuem o atributo required.

Descendente usa um espaço

CSS + HTML

```
nav a {
  color: #62748a;
}

<nav>
  <a href="/">Início</a>
  <span><a href="/ajuda">Ajuda</a></span>
</nav>
```

a dentro de nav em qualquer nível

O espaço é um combinador. Os dois links combinam porque ambos são descendentes de nav.

Leia o seletor e depois cada declaração.

Filho direto usa >

CSS + HTML

```
nav > a {
  font-weight: 700;
}

<nav>
  <a href="/">Início</a>
  <span><a href="/ajuda">Ajuda</a></span>
</nav>
```

Somente o a ligado diretamente a nav

O sinal > exige uma relação de pai e filho. O link dentro de span não combina.

Leia o seletor e depois cada declaração.

Irmãos adjacentes e irmãos posteriores

h2 + p

Seleciona o primeiro p imediatamente depois de um h2, desde que tenham o mesmo pai.

h2 ~ p

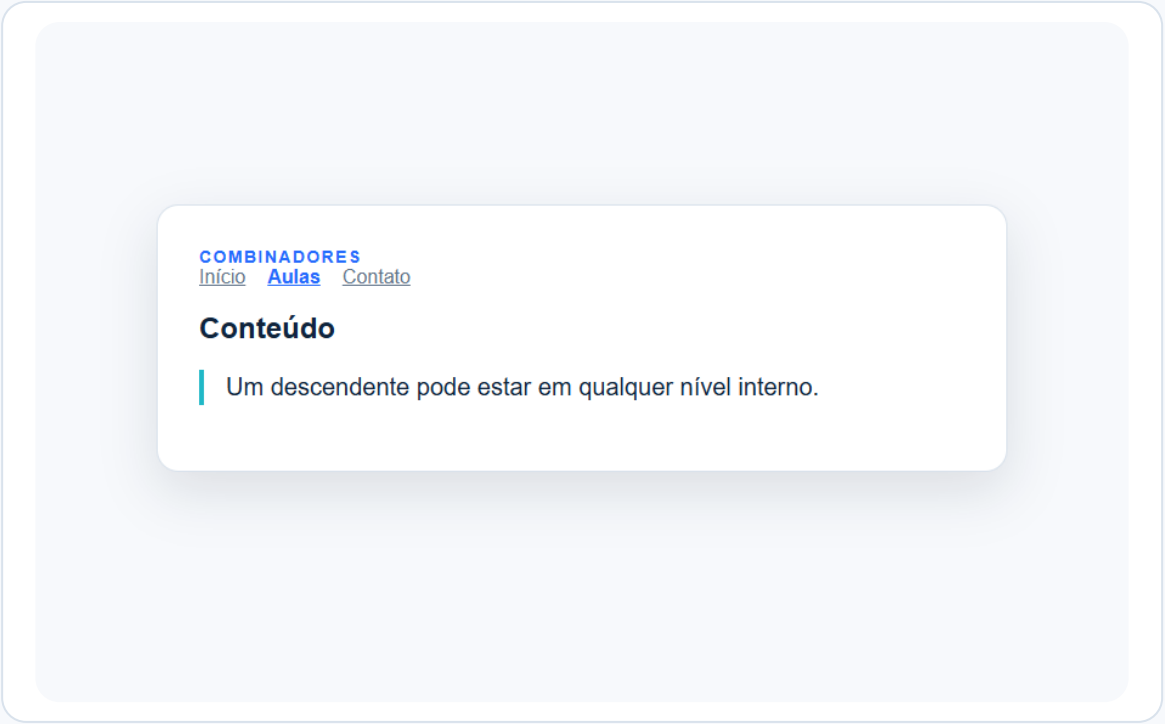
Seleciona todos os p posteriores ao h2 que compartilham o mesmo pai.

+ exige proximidade imediata; ~ aceita outros irmãos no caminho.

Combinadores no código e na tela

HTML + CSS

```
nav a { color: #62748a; }
nav > a.ativo {
  color: #246bfd;
  font-weight: 700;
}
section p {
  border-left: 4px solid #20b8c7;
}
```



O espaço indica descendente; > indica filho direto.

Pseudo-classe representa um estado

CSS

```css
a:hover {
  color: #ff6b5b;
}

input:focus {
  border-color: #246bfd;
}
```

Dois-pontos e uma condição

hover corresponde ao ponteiro sobre o elemento. focus corresponde ao elemento que recebe foco por clique ou teclado.

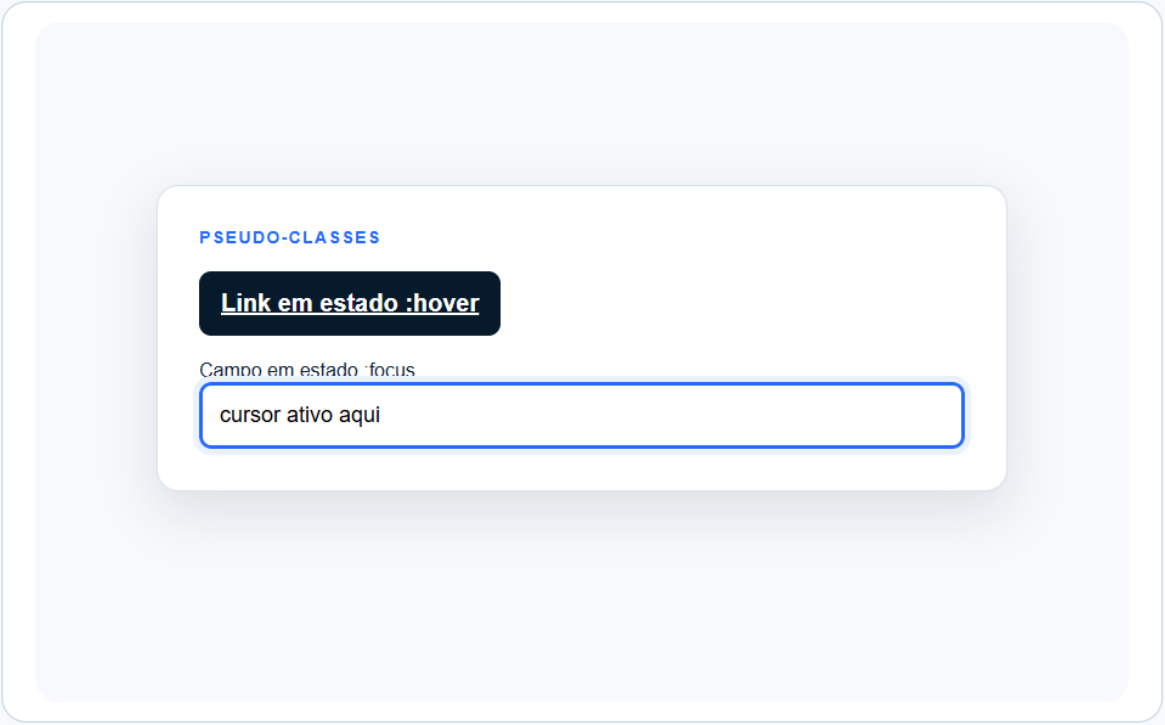
Leia o seletor e depois cada declaração.

Estados visuais precisam ser perceptíveis

HTML + CSS

```
a:hover {
  background: #071a2b;
  color: white;
}

input:focus {
  border-color: #246bfd;
  outline: 5px solid #eaf1ff;
}
```



Hover responde ao ponteiro; focus também atende navegação por teclado.

Pseudo-elemento seleciona uma parte criada ou conceitual

CSS

```
p::first-letter {
  color: #ff6b5b;
  font-size: 48px;
}

h2::after {
  content: " CSS";
}
```

Dois-pontos duplos

::first-letter estiliza a primeira letra. ::after cria uma caixa depois do conteúdo do elemento.

Leia o seletor e depois cada declaração.

Pseudo-elementos no código e na tela

HTML + CSS

```
h2::after {
  content: " CSS";
  color: #246bfd;
}

p::first-letter {
  color: #ff6b5b;
  font-size: 48px;
}
```



A apresentação muda sem adicionar novas tags para essas partes.

Resumo dos seletores

Seletor	Exemplo	Alvo
universal	`*`	qualquer elemento
tipo	`p`	todos os p
classe	`.aviso`	elementos com a classe
ID	`#principal`	elemento com o ID
atributo	`input[required]`	input com required
pseudo-classe	`a:hover`	link em um estado
pseudo-elemento	`p::first-letter`	parte do elemento

Seletores legíveis facilitam manutenção

Prefira classes com nomes que expressem função ou significado.
Evite depender de uma estrutura HTML longa e frágil.
Use ID com moderação para não elevar a especificidade sem necessidade.
Agrupe seletores quando as declarações realmente forem iguais.
Mantenha regras relacionadas próximas e com formatação consistente.

Uma página com CSS externo

Estrutura HTML, seletores reutilizáveis e resultado visível.

05

HTML do exemplo integrado

HTML

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Oficina Web</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <main class="painel">
    <p class="rotulo">SIN 493</p>
    <h1>Oficina de Web</h1>
    <p>Aprenda HTML e CSS passo a passo.</p>
    <a class="botao" href="inscricao.html">Inscreva-se</a>
  </main>
</body>
</html>
```

Estrutura e classes

link carrega o arquivo. painel, rotulo e botao identificam partes reutilizáveis. O HTML conserva texto e significado.

Leia o seletor e depois cada declaração.

CSS do exemplo integrado

CSS

```css
* { box-sizing: border-box; }
body {
  font-family: Arial, sans-serif;
  background: #f7f9fc;
  color: #10243c;
}
.painel {
  max-width: 700px;
  margin: 48px auto;
  padding: 34px;
  background: white;
}
.rotulo { color: #246bfd; font-weight: 700; }
.botao {
  display: inline-block;
  padding: 13px 22px;
  background: #246bfd;
  color: white;
  text-decoration: none;
}
```

Regras por responsabilidade

body define padrões gerais. Classes cuidam de partes específicas. A regra universal normaliza o cálculo das caixas.

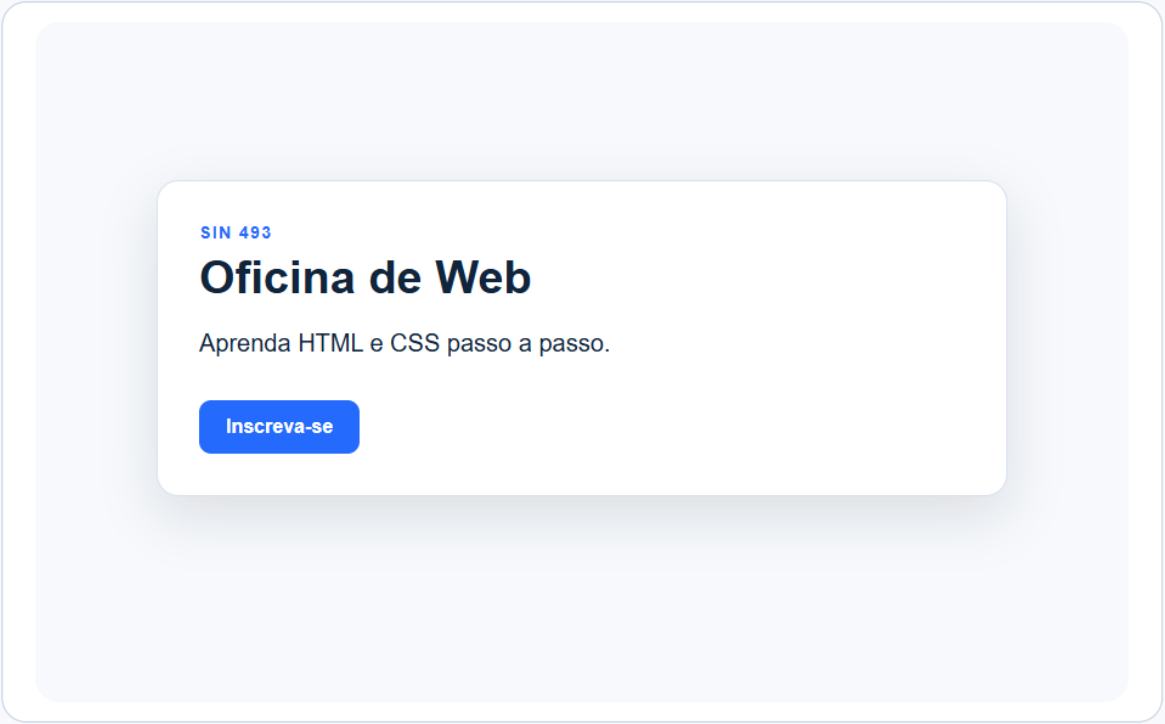
Leia o seletor e depois cada declaração.

Resultado do exemplo integrado

HTML + CSS

```
HTML
<main class="painel">
  <p class="rotulo">SIN 493</p>
  <h1>Oficina de Web</h1>
  <p>Aprenda HTML e CSS passo a passo.</p>
  <a class="botao" href="inscricao.html">Inscreva-se</a>
</main>

CSS
.painel { padding: 34px; background: white; }
.botao { background: #246bfd; color: white; }
```



O resultado nasce da combinação entre estrutura, seletores e declarações.

Atividade de aplicação

Crie index.html e estilos.css na mesma pasta.
Ligue os arquivos com <link rel="stylesheet" href="estilos.css">.
No HTML, use header, main, duas sections e footer.
Crie as classes .container, .destaque e .botao.
Use um seletor de tipo para p e um seletor de atributo para input[required].
Crie estados :hover e :focus que continuem legíveis.
Inclua duas regras concorrentes para color e explique qual vence.
Abra no navegador e registre uma captura do resultado.

Entregue os dois arquivos e uma explicação curta da cascata.

Encontre os erros no CSS

CSS COM ERROS

```
p {
  color = blue
  font-sise: 20px;
}

.destaque
  background: yellow;
}

#aviso { color: red; }
```

color usa dois-pontos, não sinal de igual.
A primeira declaração precisa de ponto e vírgula.
font-sise deve ser font-size.
.destaque precisa de chave de abertura.
Confirme se existe id="aviso" no HTML.

Explique qual regra vence

CSS + HTML

```
p { color: blue; }
.texto { color: orange; }
#principal { color: green; }

<p id="principal" class="texto"
   style="color: purple;">
  Resultado
</p>
```

As quatro declarações pertencem ao autor.
Nenhuma utiliza !important.
O atributo style está ligado diretamente ao elemento.
Resultado esperado: roxo.

Escreva os seletores pedidos

1. Todos os parágrafos — `p`
2. Elementos com classe aviso — `.aviso`
3. Elemento com ID principal — `#principal`
4. Inputs obrigatórios — `input[required]`
5. Links dentro de nav — `nav a`
6. Filhos li diretos de ul — `ul > li`
7. Link em foco — `a:focus`
8. Primeira letra do parágrafo — `p::first-letter`

Questões de fixação: fundamentos e aplicação

Fundamentos

1. Identifique seletor, propriedade e valor.
2. Para que servem chaves, dois-pontos e ponto e vírgula?
3. O que acontece com uma declaração inválida?
4. Por que separar HTML e CSS?

Formas de aplicar

5. Compare inline, interno e externo.
6. O que fazem rel e href em link?
7. Por que o CSS externo facilita manutenção?
8. Como diagnosticar um caminho incorreto?

Questões de fixação: cascata e seletores

Cascata

9. Quando a ordem de aparição decide?
10. Compare tipo, classe, ID e inline.
11. Qual é o papel de !important?
12. Diferencie cascata e herança.
13. Por que uma regra pode não entrar na disputa?

Seletores

14. Diferencie descendente e filho direto.
15. Para que serve [required]?
16. Compare :hover e :focus.
17. O que ::after cria?
18. Por que classes costumam ser mais reutilizáveis que IDs?

Síntese da Aula 07

01 **regra**

seletor + bloco

02 **declaração**

propriedade + valor

03 **inline**

no atributo style

04 **interno**

na tag style

05 **externo**

em arquivo ligado

06 **cascata**

resolve conflitos

07 **especificidade**

peso do seletor

08 **seletor**

define o alvo

> **Para entender um resultado, descubra o alvo e acompanhe a disputa.**

Mapa conceitual da Aula 07

CSS

ESCREVER	SELECIONAR	DECLARAR	RESOLVER
inline • interno • externo	tipo • classe • ID • relação	propriedade • valor	cascata • especificidade • ordem

RESULTADO NO NAVEGADOR

HTML fornece a estrutura; CSS fornece valores de apresentação

Bibliografia básica

01 COMER, Douglas E. Redes de computadores e internet. 4. ed. 2007.

02 **MEMÓRIA, Felipe. Design para a internet: projetando a experiência perfeita. 2005.**

03 **KRUG, Steve. Não me faça pensar: uma abordagem de bom senso à usabilidade Web. 2006.**

04 ALBUQUERQUE, Fernando. TCP/IP internet: programação de sistemas distribuídos: HTML, Javascript e Java. Rio de Janeiro: Axcel Books, 2001.

Bibliografia complementar

01 DEITEL, Paul J.; DEITEL, Harvey M. Ajax, rich internet applications e desenvolvimento web para programadores. São Paulo: Pearson, c2009.

02 **FERREIRA, Simone Bacellar Leal. e-Usabilidade. 2008.**

03 **NIELSEN, Jakob. Usabilidade na web. 2007.**

> **Clareza visual e previsibilidade também fazem parte da qualidade do CSS.**

W3Schools: leitura e prática

01 **W3Schools. CSS Introduction. https://www.w3schools.com/css/css_intro.asp**

02 **W3Schools. CSS Syntax. https://www.w3schools.com/css/css_syntax.asp**

03 **W3Schools. How to add CSS. https://www.w3schools.com/css/css_howto.asp**

04 **W3Schools. CSS Selectors. https://www.w3schools.com/css/css_selectors.asp**

05 W3Schools. CSS Combinators. https://www.w3schools.com/css/css_combinators.asp

06 W3Schools. CSS Pseudo-classes. https://www.w3schools.com/css/css_pseudo_classes.asp

07 W3Schools. CSS Pseudo-elements. https://www.w3schools.com/css/css_pseudo_elements.asp

08 W3Schools. CSS Specificity. https://www.w3schools.com/css/css_specificity.asp

Referências técnicas da cascata

01 **MDN Web Docs. Introduction to the CSS cascade. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascade/Introduction**

02 MDN Web Docs. Specificity. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascade/Specificity

03 MDN Web Docs. Inheritance. https://developer.mozilla.org/en-US/docs/Web/CSS/inheritance

04 MDN Web Docs. !important. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/important

05 **W3C. CSS Cascading and Inheritance Level 6. https://www.w3.org/TR/css-cascade-6/**

Dúvidas?

Quando um estilo surpreender, verifique o arquivo, o seletor e a cascata.

Próxima aula: cores, tipografia e modelo de caixas em CSS.

AULA 07